EcoLoop: Technical Implementation and Scope Validation Report

Hephzibah Ofomi | BSc. Software Engineering, African Leadership University | Supervisor: Hubert Apana | October 2026

Repository: https://github.com/H3PHZY/CAPSTONE_PROJECT.git | Walkthrough video: https://www.youtube.com/watch?v=w7FzwgZITX0 | Live app: https://ecoloop-hephzibah1.vercel.app/

1. Project Overview and Architecture

EcoLoop is a web marketplace where SME factories in Lagos's Ogba and Ikeja clusters list surplus industrial material, and nearby buyers find it, negotiate for it and pay for it. A seller uploads a photo, a Vision Transformer (ViT) model suggests one of five categories (Plastic, Glass, Metal, Biodegradable, Rubber), and the seller can override it. The app uses Next.js 16, React 19 and TypeScript, Tailwind CSS 4, Supabase (PostgreSQL, auth, storage, realtime), Hugging Face Inference, Sharp and Paystack in test mode, and is deployed on Vercel. Secret keys stay in the API routes, and row level security (RLS) limits what each user can read or change.

2. Scope Alignment and Feature Validation

Planned feature	Intended purpose	Status	Evidence
Registration and authentication	Secure access	Completed	`lib/auth-context.tsx`
Listing with photo upload	List material quickly	Completed	`QuickListingPage.tsx` (10 to 12 s, Section 4)
ViT classification with override	Cut tagging errors	Completed, with a change	`lib/classify.ts`
75% accuracy test on Lagos images	Prove local accuracy	Not completed	Planned for the pilot

Proximity filter	Find nearby material	Completed	`haversineKm` in `lib/listings.ts`
Messaging and transaction tracking	Negotiate and follow a deal	Completed	`lib/messages.ts`, `lib/transactions.ts`
Carbon estimate	Show climate benefit	Completed, one gap	`lib/carbon.ts`; the proposal's ±10% margin is not shown in the UI
Payments (deferred in proposal)	Confirm payment first	Added	`app/api/paystack/`, migration 016
Analytics, dispute handling, chat encryption, 18+ check, draft flow	Research data and trust	Not completed	Deferred
Six-week SME pilot	Test in the cluster	Not started	Scheduled for November 2026

Scope changes and why.

- Payments moved into the MVP. My supervisor asked for a sandbox payment simulation, and without payment confirmation nothing stops a lot being claimed twice, so I added Paystack in test mode.
- The code tries `google/vit-base-patch16-224` first and the specialist `yangy50/garbage-classification` model second, because the specialist often has no free hosted provider. Wood and cardboard are folded into Biodegradable to keep five categories.
- Dispute handling, chat encryption and the 18+ check were deferred because of the timeline. RLS and server-side payment checks reduce the risk for now.

3. Technical Functionality

Classification. The photo goes to Supabase Storage, then `/api/classify` checks the URL, resizes the image with Sharp and calls Hugging Face, falling back across request styles and models. Labels are mapped to a category by a lookup table and whole-word regular expressions. Results under 60% confidence are flagged, and the prediction and any seller override are saved in `classifications`.

Location. Each listing uses coordinates for the seller's industrial zone, and the distance to the viewer's cluster is calculated with the haversine formula for the radius filter. Zone centres are not GPS points, so distances are approximate.

Reservation and payment. The seller reserves the lot for one buyer (`reserve_listing_for_buyer`) and sets the price. `/api/paystack/initialize` checks the login, creates a reference (`ECO-<listing>-<timestamp>-<random>`) and returns the amount in kobo from the database. `/api/paystack/verify` asks Paystack to confirm the payment, checks the reference and amount, and calls `confirm_paystack_payment`. A deal can only be completed once paid.

Carbon. On completion, the weight is multiplied by a per-kg CO2e factor (for example Metal 4.2, Plastic 1.9) and saved in `carbon_records`. The figures are estimates, not certified offsets.

4. Testing, Performance and Compatibility

Test 1: Integration and payment verification. Why: a payment touches the browser, my API routes, Paystack and the database, so unit tests alone cannot show they agree or resist tampering. How: on the deployed Vercel app in Paystack test mode, on 30 and 31 October 2026, using three accounts in three Chrome profiles plus Brave. Deal A (₦2,000) was for reading requests and tampering, and Deal B (₦12,000) for a full payment with test card 4084 0840 8408 4081.

Case	Expected result	Actual result
Click Pay (Deal A)	Payload has only the transaction id; reference and amount come from the server	Pass. Payload was only `transactionId`; reference `ECO-ECAF5FCB-...`, 200000 kobo. A second click returned the same reference
Tampering (Deal A)	`initialize` without token: 401. Unknown deal: 404. `verify` with a made-up reference: 400	Pass. 401 'Please log in again'; 404; 400 'Payment reference was not issued for this deal'
Pay with test card (Deal B)	Payment verified and recorded	Pass. ok: true; chat showed PAID and Confirmed; `payment_ref` stored
Re-send `verify` on a finished deal	409	Pass. 409 'This deal is not reserved'

Mark completed (Deal B)	Completed, listing Sold, carbon record	Pass, except listing status. A `carbon_records` row was created. The listing stayed Live (see Improvements)

Test 2: Concurrency and lot locking. Why: two buyers must never claim the same lot, a concern raised by my supervisor. How: on the live app the Seller clicked Reserve for this buyer for two buyers in two tabs, one right after the other. The reservation function locks the listing row (`FOR UPDATE`), so the second request is rejected. I then ran a Node script (tests/concurrency/reserve-race.mjs) that sends both requests at the same instant, once.

Case	Expected result	Actual result
Manual two-tab reservation	Second request rejected	Pass. Buyer B saw 'This listing is already reserved for another buyer...'; the database had one Reserved and one Interest row
Script, simultaneous requests	One reserved, one rejected	Pass (1 run). The script printed PASS

Performance and compatibility.

Environment or metric	Result	Notes
Time to create a listing (target 60 s)	About 10 to 12 s	One person, self-timed
Lighthouse, mobile, /messages	Performance 56, Accessibility 90, Best Practices 100, SEO 63	Run without a login
Chrome (3 profiles), Brave, iPhone 16 Pro emulation	Used for testing; Paystack checkout displayed correctly	Emulated, not a real phone. Firefox, Edge and real phones not tested

Improvements after testing. The listing still showed Live after a reservation or completion, because migration `002_transaction_listing_sync.sql` (the trigger that updates the listing status) had not been applied to

the live Supabase project. I applied it, and a lot reserved afterwards showed Reserved. Still to do: a Paystack webhook, so a payment is recorded even if the buyer closes the browser early.

5. Code Quality, OOP Principles and Algorithms

Structure. `app/` holds routes and API endpoints, `components/` the interface by feature, `lib/` data access and logic one concern per file, and `supabase/migrations/` 16 SQL files that rebuild the database. Secrets and payment logic stay on the server, and the rules that must not break (one reserved buyer per lot, no completion without payment) are enforced in the database as well as the interface. The code uses TypeScript modules and React components, not class hierarchies.

Algorithms. Haversine distance for the radius filter; label normalisation with a lookup, whole-word regular expressions and a 60% confidence flag; a fallback chain across request styles and models; and atomic reservation and payment in PostgreSQL (migration 016), using `FOR UPDATE` row locks, a trigger that blocks a second reserved buyer, and a server-issued reference with a 30 minute expiry.

Limitations. `MessagesPage.tsx` is about 1,000 lines and `lib/classify.ts` repeats similar request code, so I plan to split them and put the classifiers behind one interface. There are no automated unit tests yet.

6. Conclusion

EcoLoop delivers the full proposed MVP and adds sandbox payments with database-level protection against double claims. The main gaps are the local accuracy test, pilot analytics and the deferred trust features, planned for the pilot phase.